DECA Lab Logbook — Spring Term

Lab 1 | Task 7: SHIFT Instructions

Task 7: SHIFT Instructions

Objectives

- How shift commands (LSL, LSR, ASR, XSR) are encoded in the EEP1 instruction set
- How these shift commands are executed using a barrel shifter and the associated shifter hardware

```
MOV R2, #0x1
LSL R2, R2, #1
LSL R2, R2, #2
LSL R2, R2, #4
LSL R2, R2, #8
ASR R2, R2, #3
LSR R2, R2, #6
ASR R2, R2, #6
MOV R2, #0xFF
ADD R0, #0   // FLAGC := 0
XSR R2, R2, #3
SUB R0, R0, R0   // FLAGC := 1
XSR R2, R2, #2
```

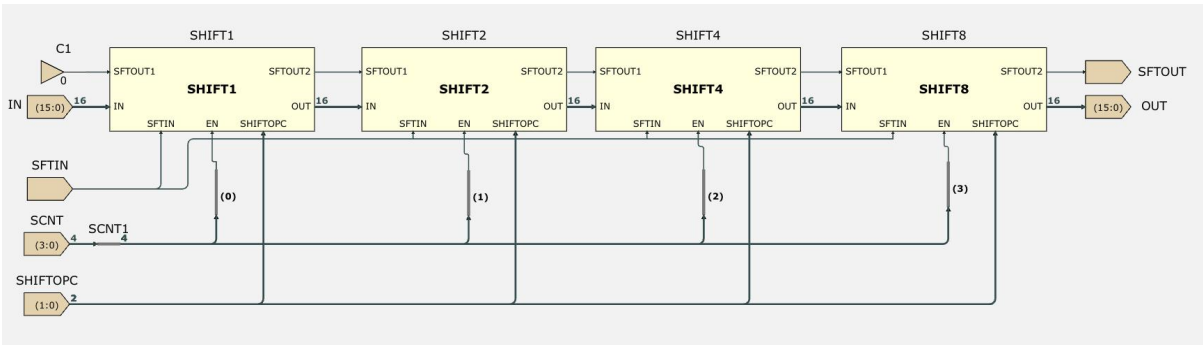
1: Open the shift sheet. Using your tracing skills and the EEP1 Instruction Encoding sheet, deduce the meaning of each signal on the ports of the shift block. You do not need to understand the internals of each individual shifter (SHIFT1, SHIFT2, etc.) in detail; you may assume that each shifter: • shifts its input by a fixed number of bits when its enable signal EN is 1; • passes its input through unchanged when EN is 0; • uses SHIFTOPC to select which of the four shift behaviours (LSL, LSR, ASR, XSR) it performs. Explain how the combination of SHIFT1, SHIFT2, SHIFT4, and SHIFT8 allows the barrel shifter arrangement to perform the correct shift (type and amount) given the shift count (SCNT) encoded in the instruction.

Signal	Width	Meaning
IN	16	Input data (from RA)
SCNT	4	Shift count from bits[3:0] of the instruction word; encodes amounts 0–15
SHIFTOPC	2	Shift type: 00 = LSL, 01 = LSR, 10 = ASR, 11 = XSR
SFTIN	1	Carry-in for XSR (value of FlagC)
SFTOUT	1	Last bit shifted out; written to FlagC
OUT	16	Shifted result

The four stages are cascaded in series. Each stage is enabled by the corresponding bit of SCNT: SHIFT1.EN = SCNT[0], SHIFT2.EN = SCNT[1], SHIFT4.EN = SCNT[2], SHIFT8.EN = SCNT[3]. When EN = 1 the stage shifts by its fixed amount; when EN = 0 it passes data unchanged. The total shift is the sum of the amounts of all enabled stages, equal to the binary value of SCNT. SHIFTOPC propagates consistently through all active stages, so the shift type is applied correctly regardless of the count.

Waveform: starting from R2 = 1, LSL ×1/2/4/8 gives 2 → 8 → 128 → 32768 (= −32768 signed, since bit 15 = 1). ASR #3 on 0x8000 → 0xF000 = −4096 (sign replicated). LSR #6 on 0xF000 → 0x03C0 = 960 (zeros fill). XSR #3 with FLAGC = 0 on 0xFFFF → 0x1FFF = 8191; then XSR #2 with FLAGC = 1 → 0xC7FF = −14337.

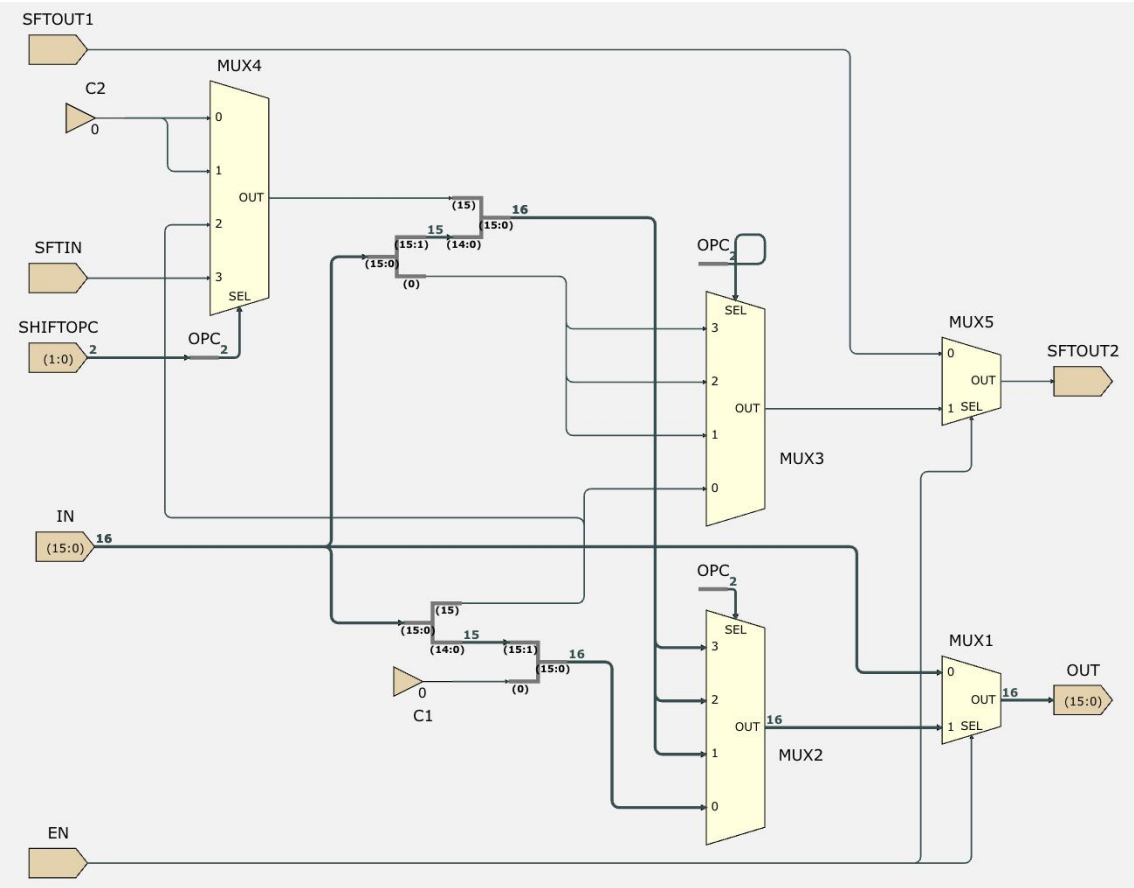
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Reg16x8.R0(15:0)	0														0
Reg16x8.R1(15:0)	0														0
Reg16x8.R2(15:0)	0	1	2	8	128	-32768	-4096	960	15	-1	-1	8191		-14337	
Reg16x8.R3(15:0)	0														0
Reg16x8.R4(15:0)	0														0



2: Open the SHIFT1 sheet and explain how its hardware implementation enables it to: • shift the input data by 1 bit in the required direction; • implement all four shift types (LSL, LSR, ASR, XSR) with appropriate carry-in and carry-out behaviour.

- Left-shift path: IN(14:0) → OUT(15:1). MUX4 supplies the shifted-in bit at OUT(0): constant 0 for LSL.
- Right-shift path: IN(15:1) → OUT(14:0). MUX4 supplies the shifted-in bit at OUT(15): constant 0 for LSR; IN(15) for ASR (sign-bit replication); SFTIN for XSR (carry flag).
- MUX2: selects left or right candidate result based on SHIFTOPC (LSL → left; LSR/ASR/XSR → right).
- MUX1 (controlled by EN): EN = 0 bypasses the stage and passes IN unchanged; EN = 1 outputs the shifted result.
- MUX3/MUX5 generate SFTOUT: MSB for left shifts, LSB for right shifts — the bit shifted out, written to FlagC.

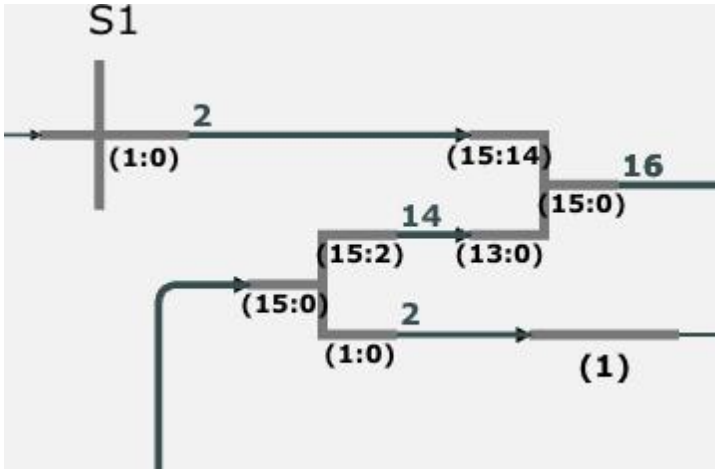
The sheet diagram is on the next page.



3: Compare the other three shifter sheets (SHIFT2, SHIFT4, SHIFT8) with SHIFT1. Describe how they differ so that they shift by 2, 4, or 8 bits respectively, while otherwise implementing the same four shift types. In particular, explain how the carry-in behaviour of ASR is implemented correctly when the shift amount is greater than 1, and why this is essential for its intended meaning.

SHIFT2, SHIFT4, and SHIFT8 are structurally identical to SHIFT1, differing only in the number of bit positions wired in the shift paths (2, 4, or 8 instead of 1).

ASR sign-bit propagation: in each stage, MUX4 selects the current input's bit 15 as the shifted-in bit for ASR. After SHIFT1, the output's MSB is the original sign bit. SHIFT2 then reads that MSB and fills the two new MSB positions with it. This chain ensures that all vacated positions are filled with the original sign bit, which is essential for ASR to implement arithmetic right shift — incorrect sign-bit filling would produce wrong results for any shift greater than 1.



Extended questions

1: Inspect the multiplexer that selects the shifted-in bit for each shift block. What limitation of the XSR instruction does this hardware arrangement reveal?

MUX4 can only select the shifted-in bit from a fixed set: constant 0, the MSB (IN(15)), or SFTIN (FlagC). XSR is therefore constrained to use the single-bit carry flag as its fill value. It cannot inject arbitrary bit patterns or values from other sources — XSR is an extended shift only in the sense of using the carry flag, not a fully general arbitrary-fill shift.

2: Why is the 1/2/4/8 barrel shifter more efficient than using sixteen SHIFT1 blocks in series? Could you design a barrel shifter that way, and what would be the disadvantages?

Sixteen SHIFT1 blocks in series would achieve shift amounts 0–15, but with significant disadvantages: the critical path passes through up to 15 MUX1 stages even when most are in bypass mode, adding substantial propagation delay. The 1/2/4/8 design has a critical path through at most 4 stages. It also uses roughly one-quarter the hardware area and power. For a shift of 8 bits, the logarithmic design activates a single SHIFT8 stage; the series design would propagate through seven bypass stages and one active stage — more logic, more delay, and no benefit. The powers-of-2 stage arrangement achieves the same range of shifts in O(log N) hardware and delay.